

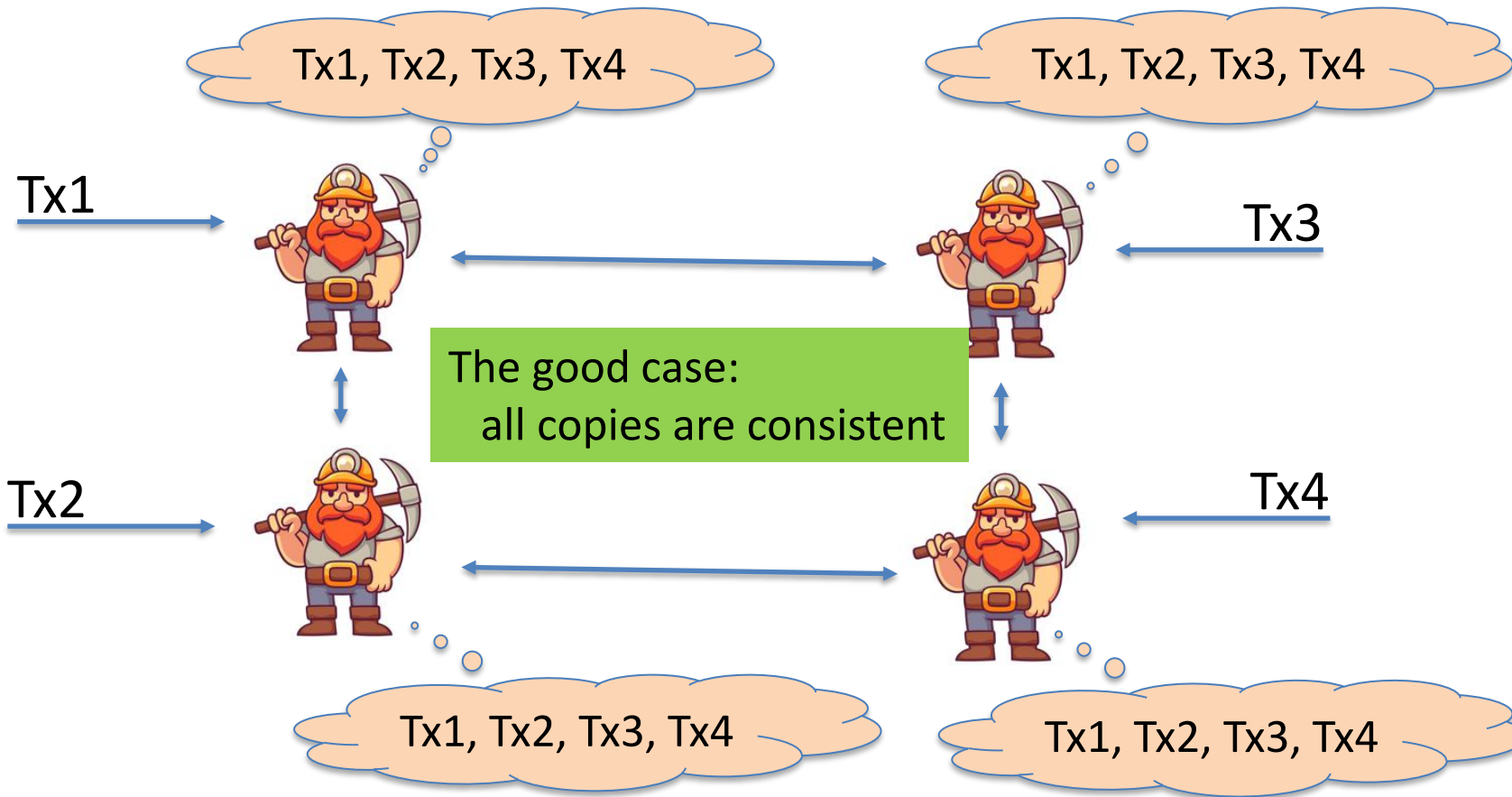
Blockchain: Smart Contracts

Lecture 4

Fundamentals of Consensus

The goal of consensus

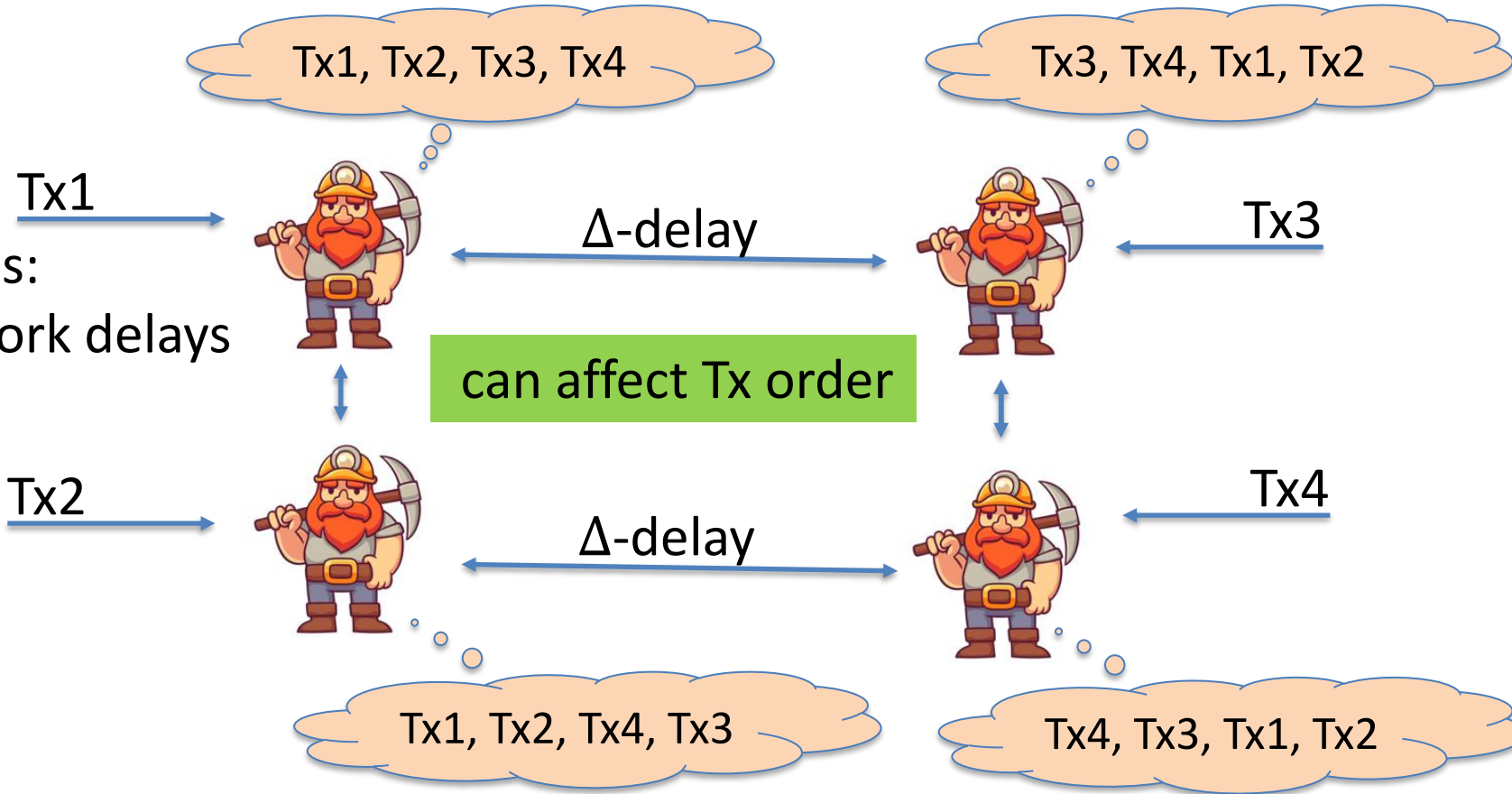
(informally for now)



Why is consensus a hard problem?

Problems:

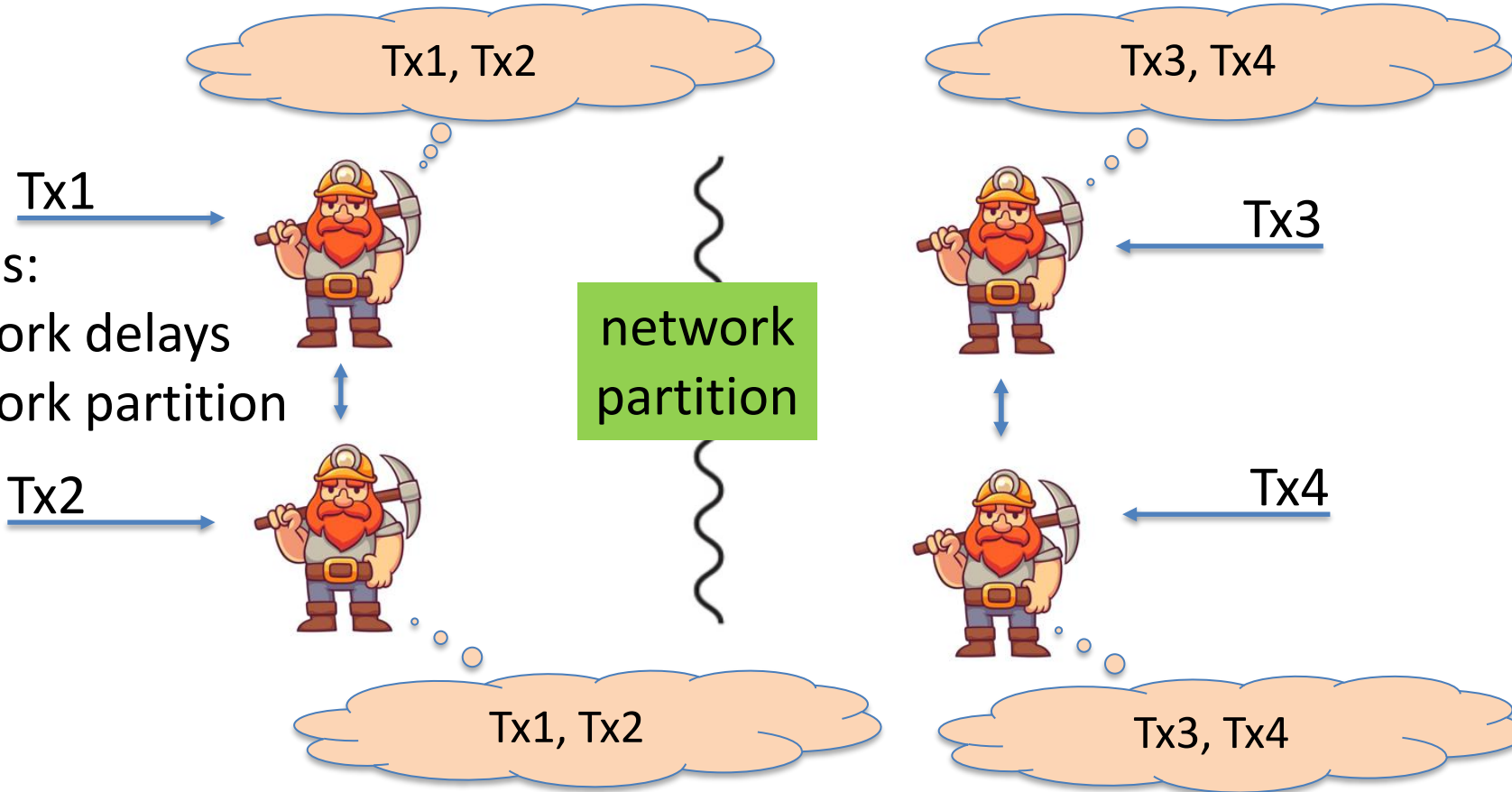
- Network delays



Why is consensus a hard problem?

Problems:

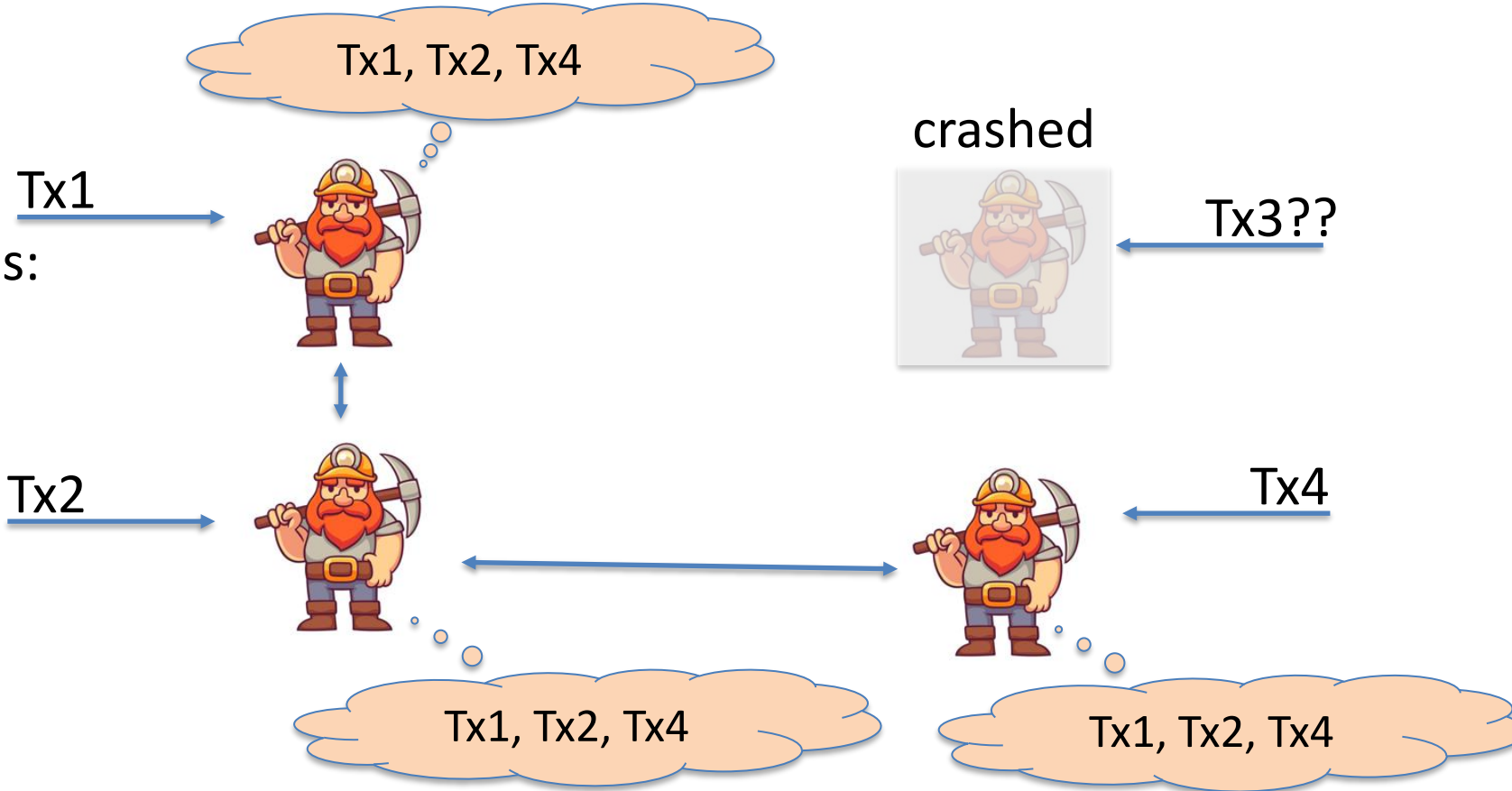
- Network delays
- Network partition



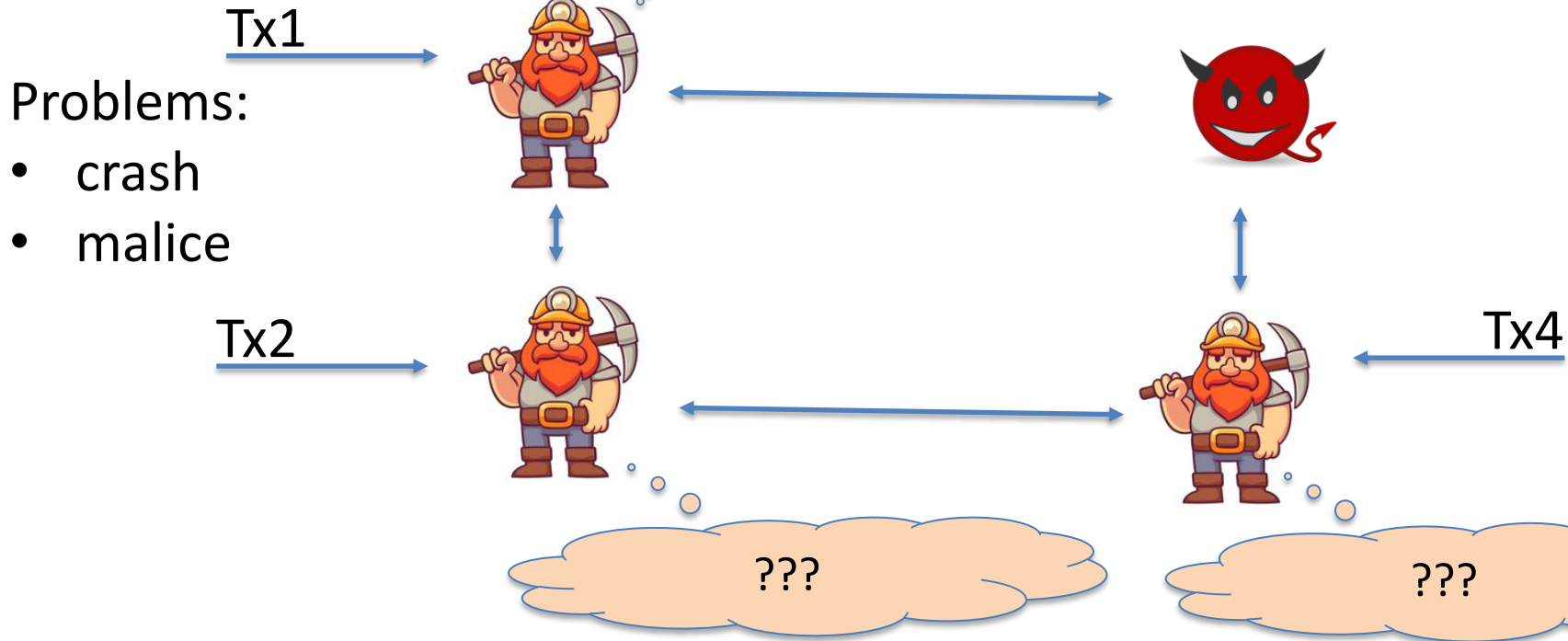
Why is consensus a hard problem?

Problems:

- crash



Why is consensus a hard problem?



Let's get started ...

The Byzantine Generals Problem

LESLIE LAMPORT, ROBERT SHOSTAK, and MARSHALL PEASE
SRI International

July 1982

Reliable computer systems must handle malfunctioning components that give conflicting information to different parts of the system. This situation can be expressed abstractly in terms of a group of generals of the Byzantine army camped with their troops around an enemy city. Communicating only by messenger, the generals must agree upon a common battle plan. However, one or more of them

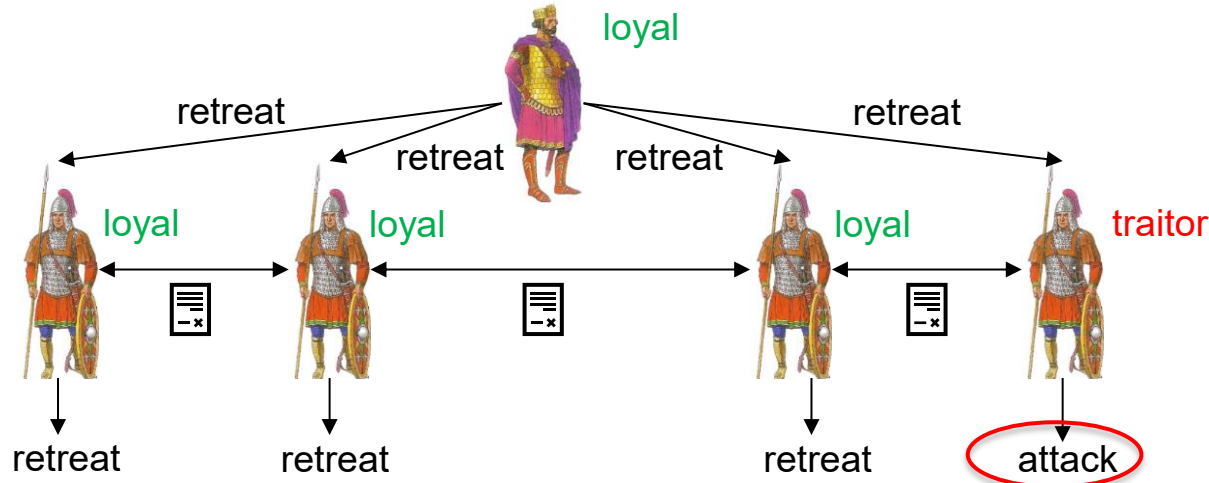
The Byzantine Generals Problem

- Encapsulates the problem of reaching consensus.
- **Problem statement:**
 - There are n generals (where n is fixed), one of which is the *commander*.
 - Some generals are *loyal*, and some of them can be *traitors* (including the commander). Nodes can become traitors mid-game.
 - The commander sends out an order that is either *attack* or *retreat* to each general.
 - If the commander is *loyal*, it sends the *same* order to all generals.
 - All generals take an action (*attack* or *retreat*) after some time.

Byzantine Generals Problem

Goal:

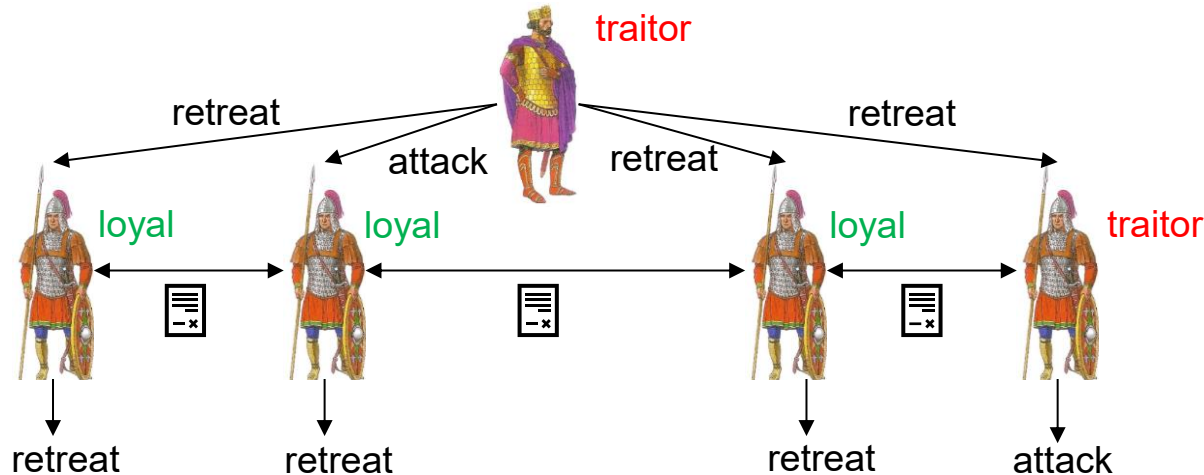
- **Agreement:** No two **loyal** generals take **different** actions.
- **Validity:** If the commander is **loyal**, then all **loyal** generals must take the action suggested by the commander.
- **Termination:** All **loyal** generals must eventually take some action.



Byzantine Generals Problem

Goal:

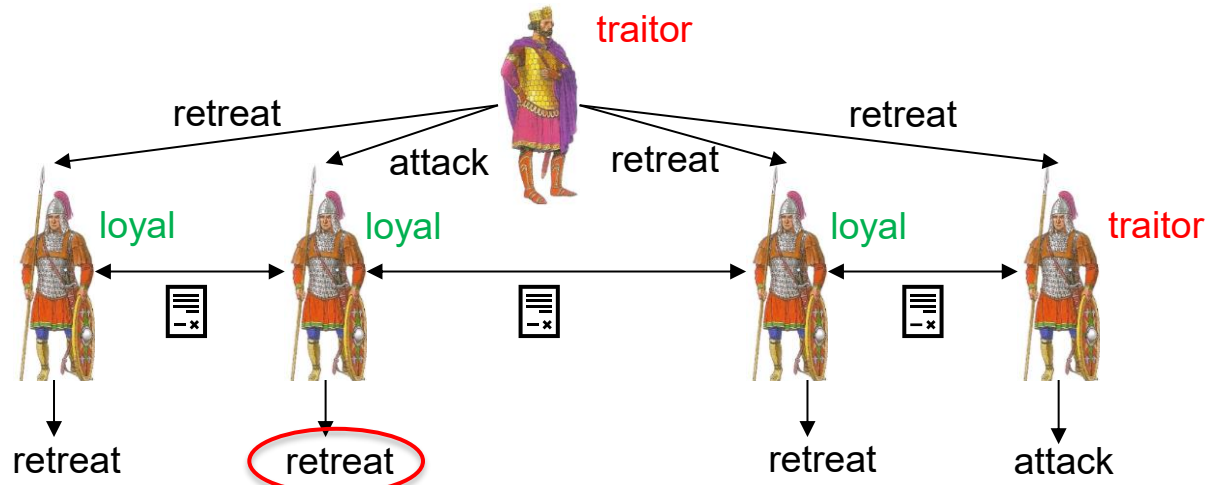
- **Agreement:** No two **loyal** generals take **different** actions.
- **Validity:** If the commander is **loyal**, then all **loyal** generals must take the action suggested by the commander.
- **Termination:** All **loyal** generals must eventually take some action.



Byzantine Generals Problem

Goal:

- **Agreement:** No two **loyal** generals take **different** actions.
- **Validity:** If the commander is **loyal**, then all **loyal** generals must take the action suggested by the commander.
- **Termination:** All **loyal** generals must eventually take some action.



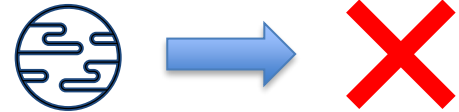
From Generals to Nodes

- Solution to the Byzantine Generals Problem is a *consensus protocol*.
- When modelling consensus protocols:
 - Generals → Nodes
 - Commander → Leader
 - Loyal → Honest, Traitor → Adversary
 - What can the adversarial nodes do?

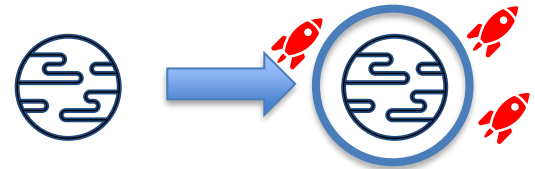
Adversary



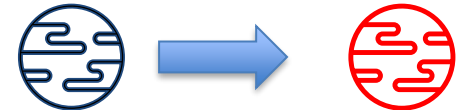
- *The adversary can corrupt nodes, after which they are called **adversarial**.*
 - **Crash faults** if the adversarial nodes do not send or receive any messages.



- **Omission faults** if the adversarial nodes can selectively choose to drop or let through each message sent or received.



- **Byzantine faults (Byzantine adversary)** if the adversarial nodes can deviate from the protocol arbitrarily.



Adversary



We typically bound the adversary's power by assuming an upper bound (f) on the number of nodes (n) that can ever be adversarial.

- e.g., $f < n$, $f < \frac{n}{2}$, $f < \frac{n}{3}$, ...

Communication



- Nodes can send messages to each other, authenticated by *signatures*.
- There is a public key infrastructure (PKI) setup.
 - Adversary cannot simulate honest nodes! (i.e. control honest nodes)
 - There are other ways to prevent such simulation (e.g., proof-of-work).

Consensus protocols typically assume that the adversary cannot forge signatures of honest nodes.

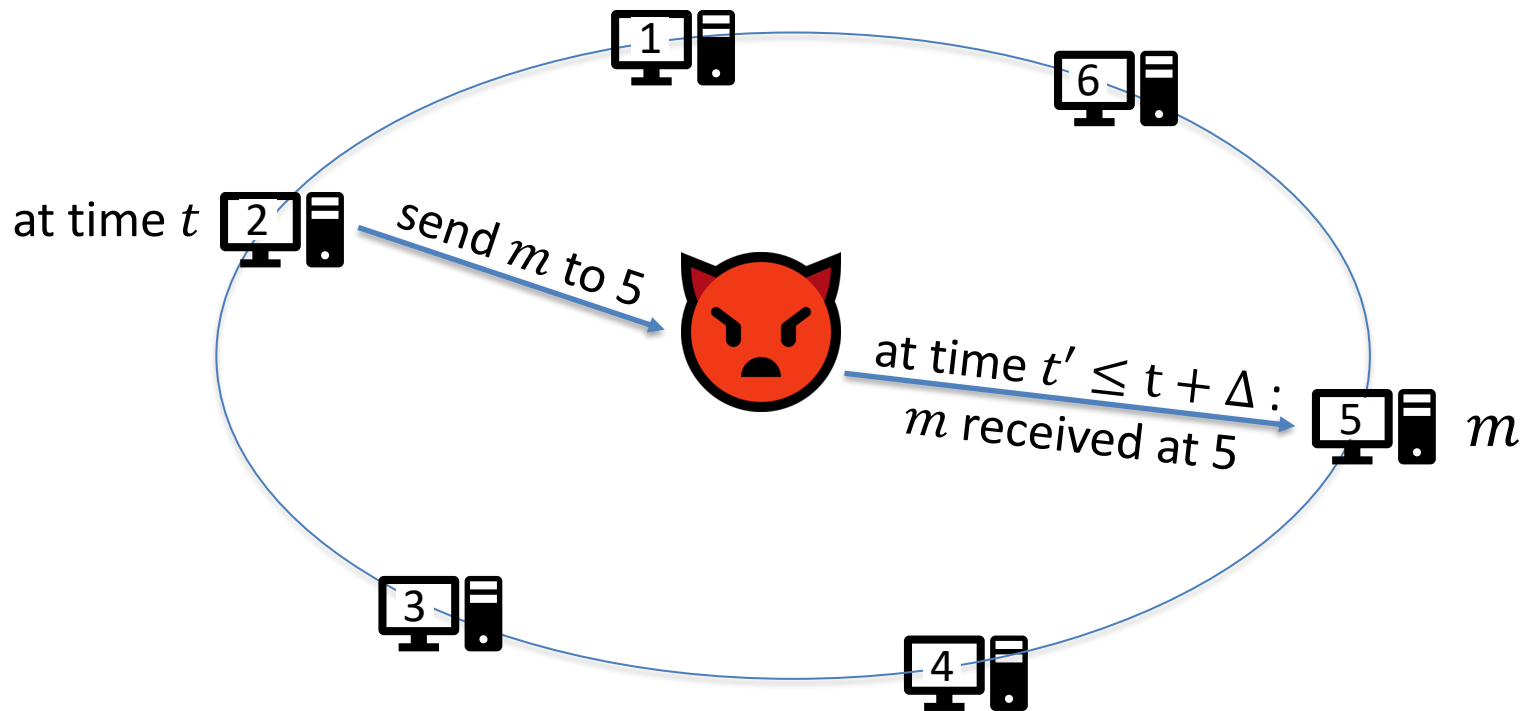
Network Models

We assume that the adversary *controls* the delivery of the messages subject to certain limits (the adversary runs the network):

- In a **synchronous network**, adversary must deliver any message sent by an honest node to its recipient(s) within Δ rounds. Here, Δ is a *known* bound.
- In an **asynchronous network**, adversary can delay any message for an arbitrary, yet finite amount of time. However, it must eventually deliver every message sent by the honest nodes.

A synchronous network (abstractly)

The adversary runs the network, subject to Δ -synchronicity constraint



Byzantine Generals Problem

- There are n generals (where n is fixed), one of which is the commander.
- For a public f , a subset of f generals is adversarial, and all other generals are loyal.
- The commander sends out an order that is either attack or retreat to each general.
- Network is synchronous.

Byzantine Generals Problem:

- **Agreement:** No two **loyal** generals take **different** actions.
- **Validity:** If the commander is **loyal**, then all **loyal** generals must take the action suggested by the commander.
- **Termination:** All **loyal** generals must eventually take some action.

Byzantine Broadcast (BB)

- There are n nodes (where n is fixed), one of which is the leader.
- For a public f , a subset of f nodes is adversarial, and all other nodes are honest
- The leader has an input value 0 or 1.
- Network is synchronous.

Byzantine Broadcast Problem:

- **Agreement:** No two **honest** nodes output **different** values.
- **Validity:** Leader is **honest** $\Rightarrow$ All **honest nodes** output the value **input to the leader**.
- **Termination:** All **honest** nodes eventually output some value.

Byzantine Broadcast (BB)

- There are n nodes (where n is fixed), one of which is the leader.
- For a public f , a subset of f nodes is adversarial, and all other nodes are honest
- The leader has an input value 0 or 1.
- Network is synchronous.

Byzantine Broadcast Problem:

- **Agreement:** No two **honest** nodes output **different** values.
- **Validity:** Leader is **honest** $\Rightarrow$ All **honest nodes** output the value **input to the leader**.
- **Termination:** All **honest** nodes eventually output some value.

even when the leader
is adversarial!!

Byzantine Broadcast (BB)

- There are n nodes (where n is fixed), one of which is the leader.
- For a public f , a subset of f nodes is adversarial, and all other nodes are honest
- The leader has an input value 0 or 1.
- Network is synchronous.

Byzantine Broadcast Problem:

- **Agreement:** No two **honest** nodes output **different** values.
- **Validity:** Leader is **honest** $\Rightarrow$ All **honest nodes** output the value **input to the leader**.
- **Termination:** All **honest** nodes eventually output some value.

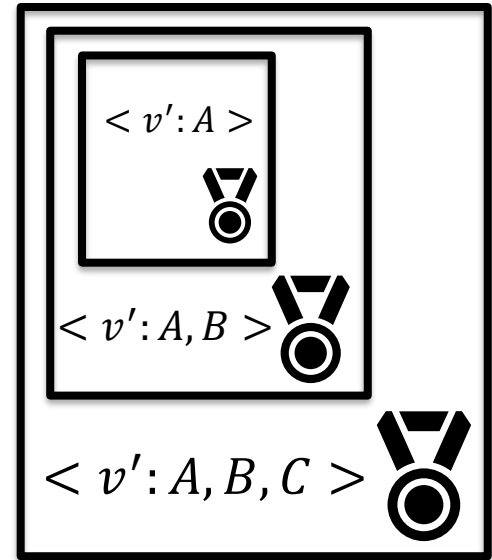
No double
spend

even when the leader
is adversarial!!

No
censorship

Protocol for BB: Setup

- Denote the nodes by the letters $i \in \{A, B, C, D, \dots\}$
- Node A is the leader. Let v denote its input value.
- Let V_i denote the set of values received by node i .
- Time moves in *lock-step*.



- Let $\langle v': i \rangle$ denote a value v' that is signed by node i .
- Let $\langle v': i, j, \dots, l, k \rangle$ denote a *signature chain* signed by $i, j, \dots, l, k$:
Recursive definition: $\langle v': i, j, \dots, k \rangle = \langle \langle v': i, j, \dots, l \rangle : k \rangle$

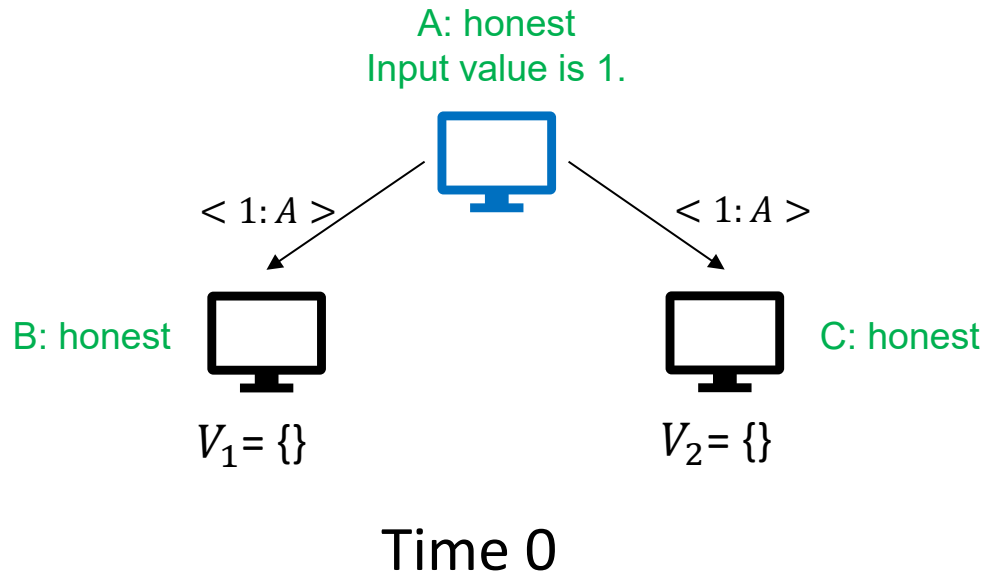
Strawman Protocol I (assuming $\Delta = 1$)

- Time 0: Leader broadcasts $\langle v:A \rangle$. // v is either 0 or 1.
(the broadcast value)
- Time 1:
 - Node i :
 - Upon receiving any $\langle v':A \rangle$, add v' to V_i .
 - Decide value choice(V_i).

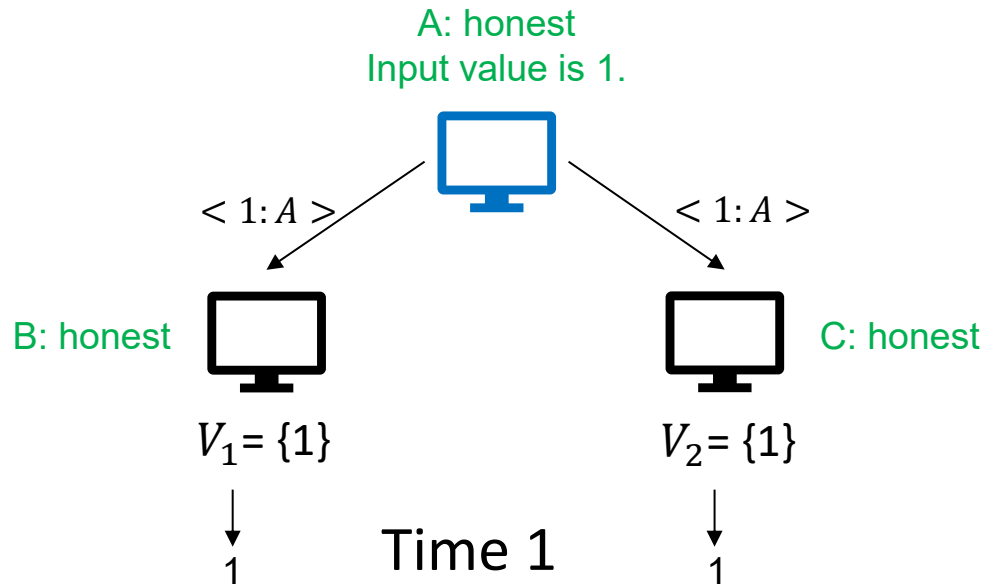
choice(V_i):

- If $V_i = \{v\}$, return v . // if V_i contains only one value, return it
- Else, return 0. // if V_i contains more than one value, return 0

Strawman Protocol I



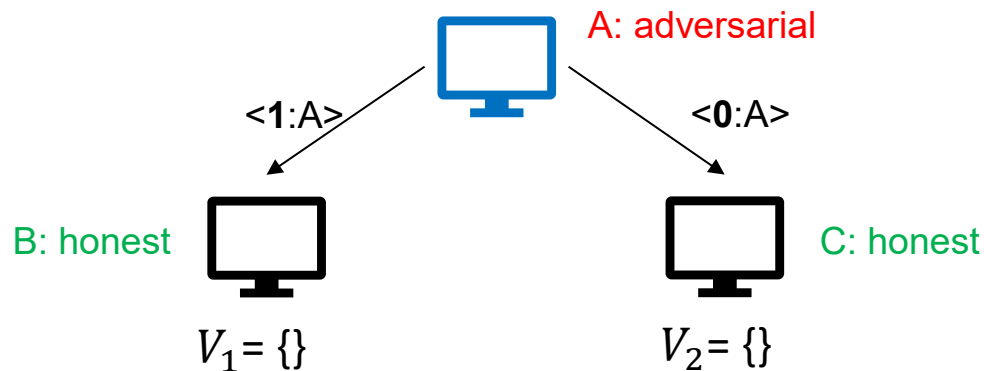
Strawman Protocol I



Validity is satisfied!

Strawman Protocol I

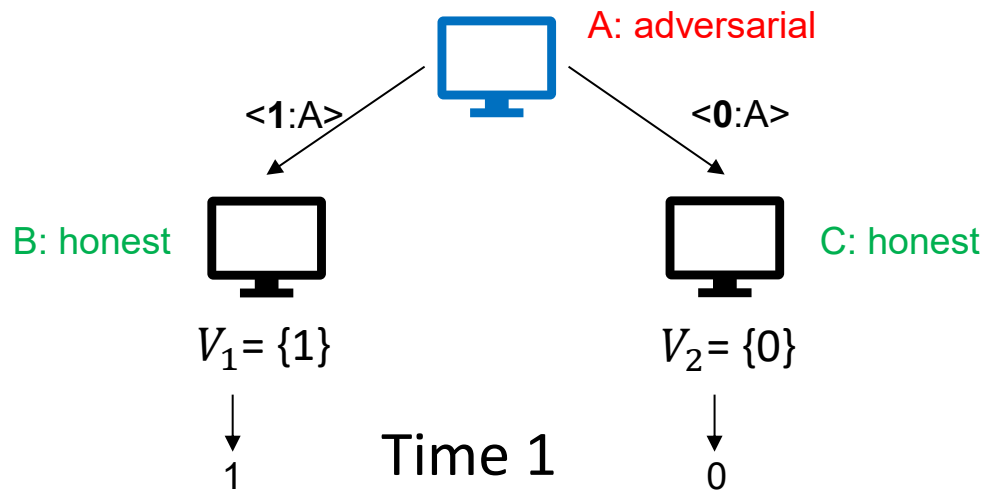
Problem: what if the leader is adversarial?



Time 0

Strawman Protocol I

Problem: what if the leader is adversarial?

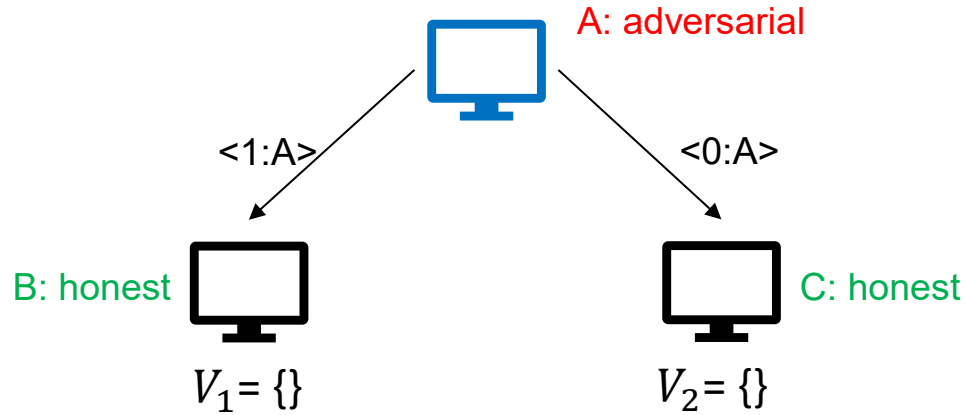


Agreement is violated!

Strawman Protocol II

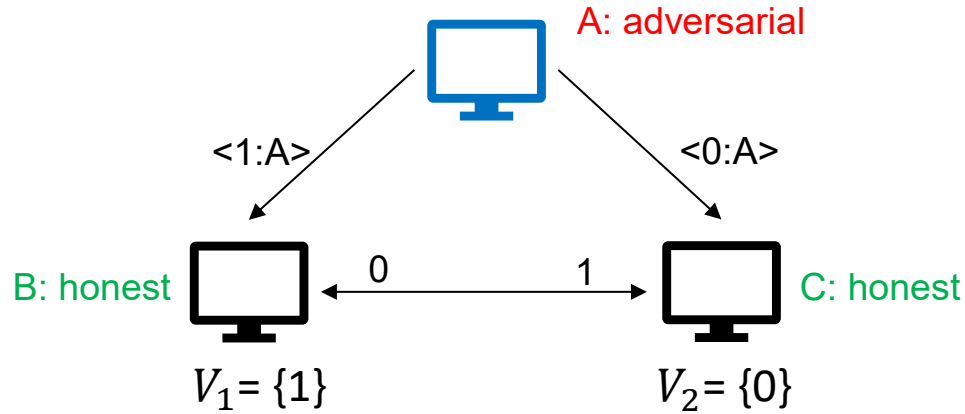
- Time 0: Leader broadcasts $\langle v:A \rangle$. // v is either 0 or 1.
(the broadcast value)
- Time 1:
 - Node i :
 - Upon receiving any $\langle v':A \rangle$,
add v' to V_i ,
and broadcast $\langle v':A,i \rangle$.
- Time 2:
 - Node i :
 - Upon receiving any $\langle v':A,j \rangle$, where $j \neq A$, add v' to V_i .
 - Decide value choice(V_i).

Strawman Protocol II



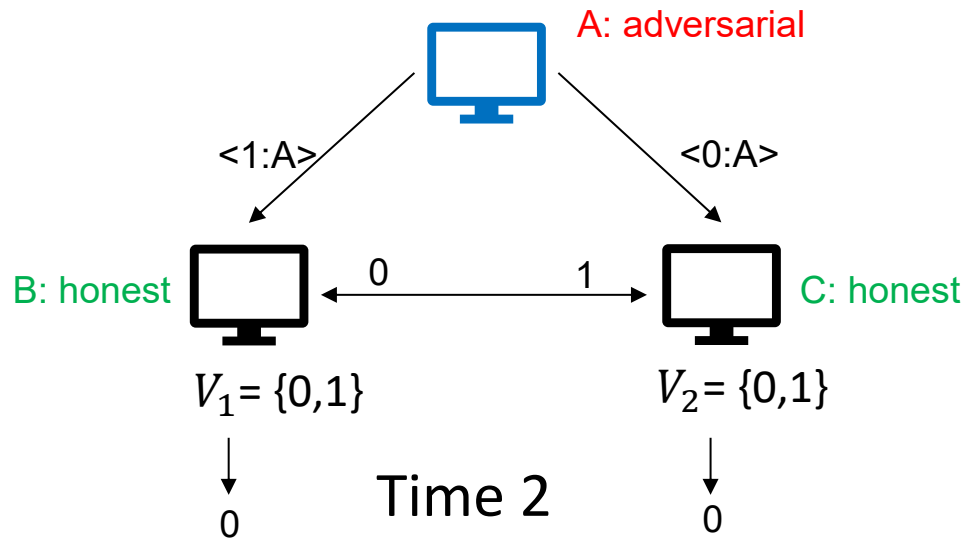
Time 0

Strawman Protocol II



Time 1

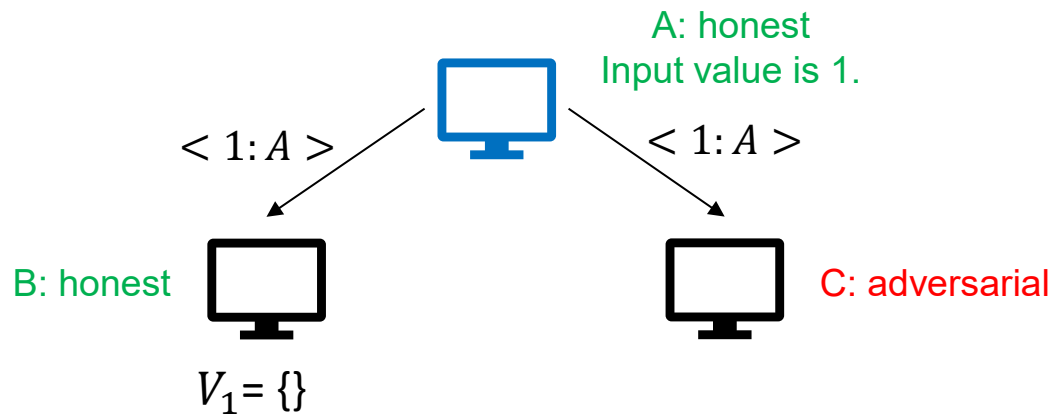
Strawman Protocol II



Agreement is satisfied!

Strawman Protocol II

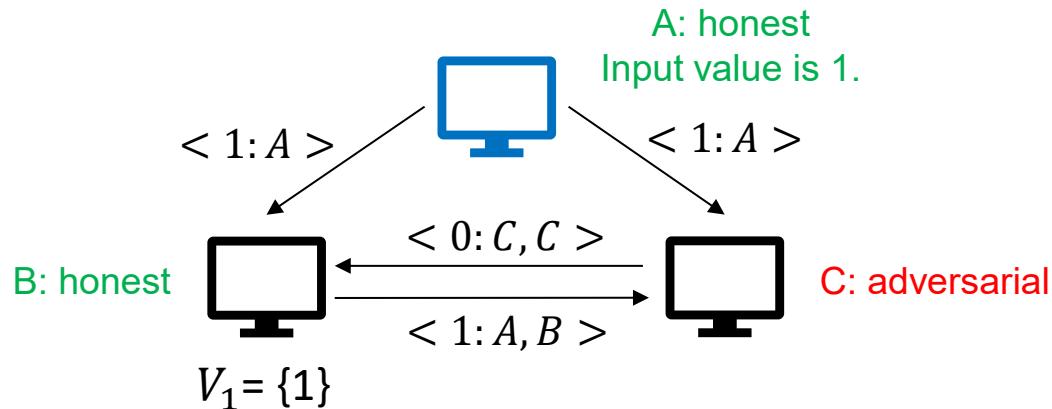
Problem: what if one of the nodes is adversarial?



Time 0

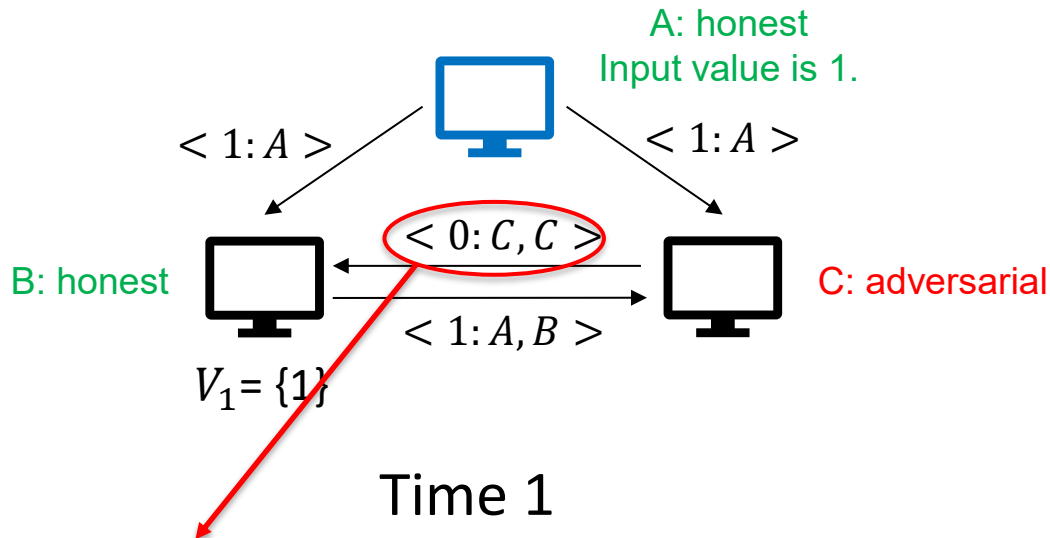
Strawman Protocol II

Problem: what if one of the nodes is adversarial?



Strawman Protocol II

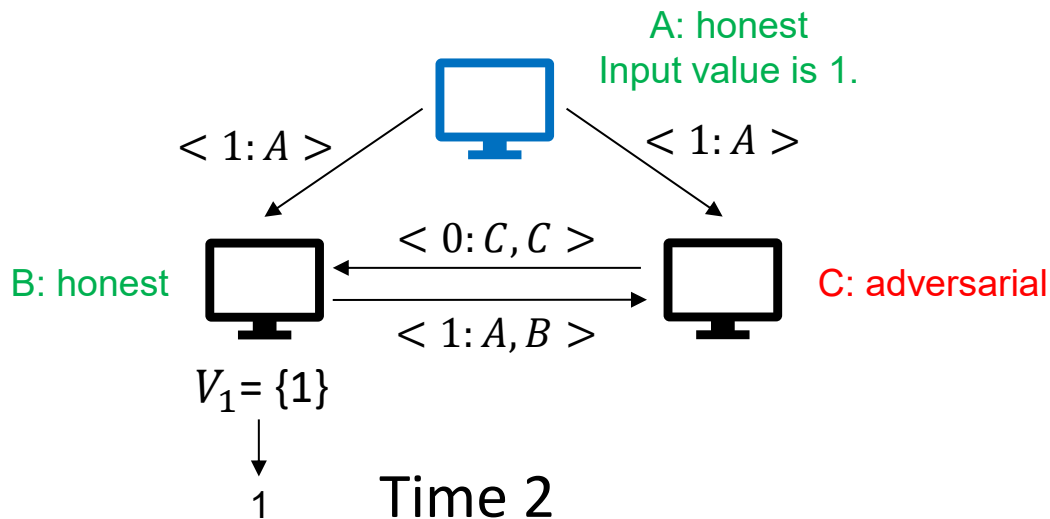
Problem: what if one of the nodes is adversarial?



Invalid since the first signature is not by the leader, i.e., node A.
Thus, 0 is not added to V_1 .

Strawman Protocol II

Problem: what if one of the nodes is adversarial?



Validity is satisfied as well!
So are agreement and termination!

Dolev-Strong (1983)

- Time 0: Leader broadcasts $\langle v: A \rangle$. // v is either 0 or 1.
(the broadcast value)
- Time $t = 1, \dots, f$:
 - Node i :
 - Upon receiving any $\langle v': A, i_1 \dots, i_{t-1} \rangle$, where $i \neq i_1 \neq \dots \neq i_{t-1}$ and $v' \notin V_i$, add v' to V_i and broadcast $\langle v': A, i_1 \dots, i_{t-1}, i \rangle$.
- Time $f + 1$:
 - Node i :
 - Upon receiving any $\langle v': A, i_1 \dots, i_f \rangle$, where $i \neq i_1 \neq \dots \neq i_f$ and $v' \notin V_i$, add v' to V_i .
 - Decide value choice(V_i).

Security of Dolev-Strong (1983)

Theorem (Dolev-Strong, 1983): For any $f < n$, Dolev-Strong (1983) with n nodes and $f + 1$ rounds satisfies agreement, validity and termination in a synchronous network.

Converse Theorem: Any (deterministic) protocol that satisfies agreement, validity and termination for n nodes in a synchronous network with resilience up to f crash (as well as Byzantine) faults must have an execution with at least $f + 1$ rounds.

State Machine Replication

A Repeated Byzantine Generals Problem

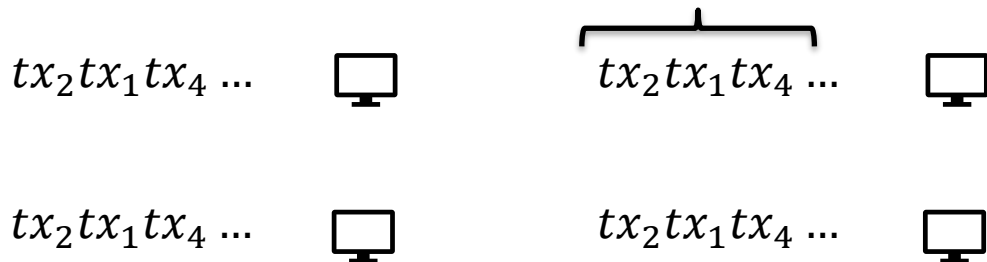
State Machine Replication (SMR)

A Centralized Bank



Blockchain (State Machine Replication)

Log (Ledger): an ever-growing, linearly-ordered *sequence* of transactions.

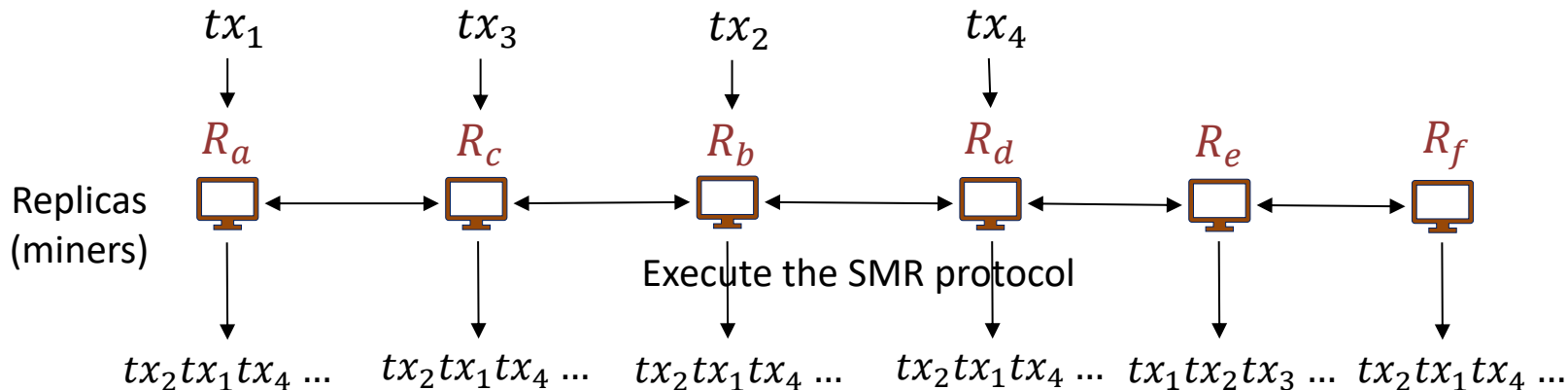


State Machine Replication (SMR)

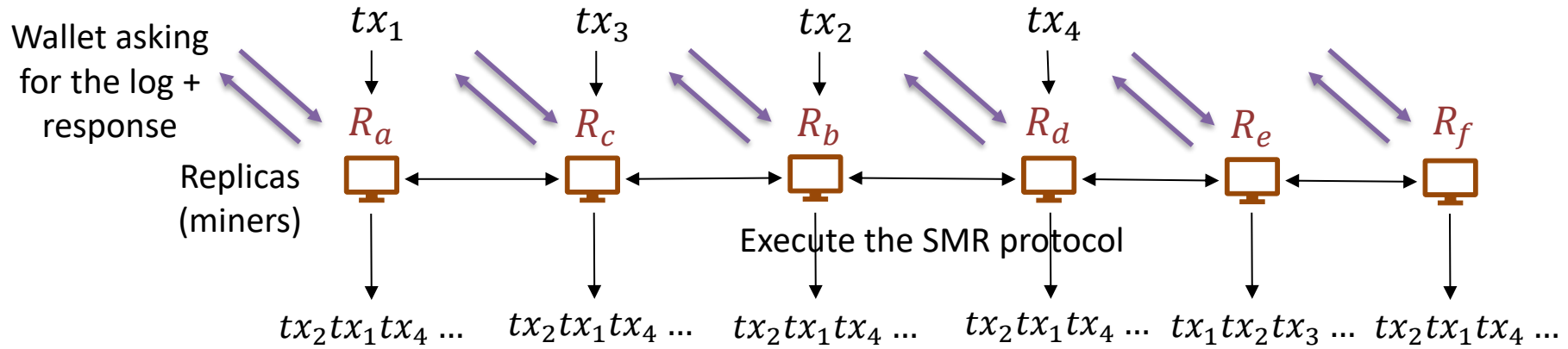
Two parties of SMR:

- *Replicas* receive transactions, execute the SMR protocol and determine the log.
- *Clients* are the learners: They communicate with the replicas to learn the log.

Goal of SMR is to ensure that the *clients* learn the *same* log.



State Machine Replication (SMR)



$$LOG_t^1 = tx_2tx_1tx_4 \dots$$



Wallets are an example of a client.

$$LOG_t^2 = tx_2tx_1tx_4 \dots$$



Clients (Wallets)

Wallets ask the replicas what the correct log is.

Clients (Wallets)

$$LOG_t^3 = tx_2tx_1tx_4 \dots$$

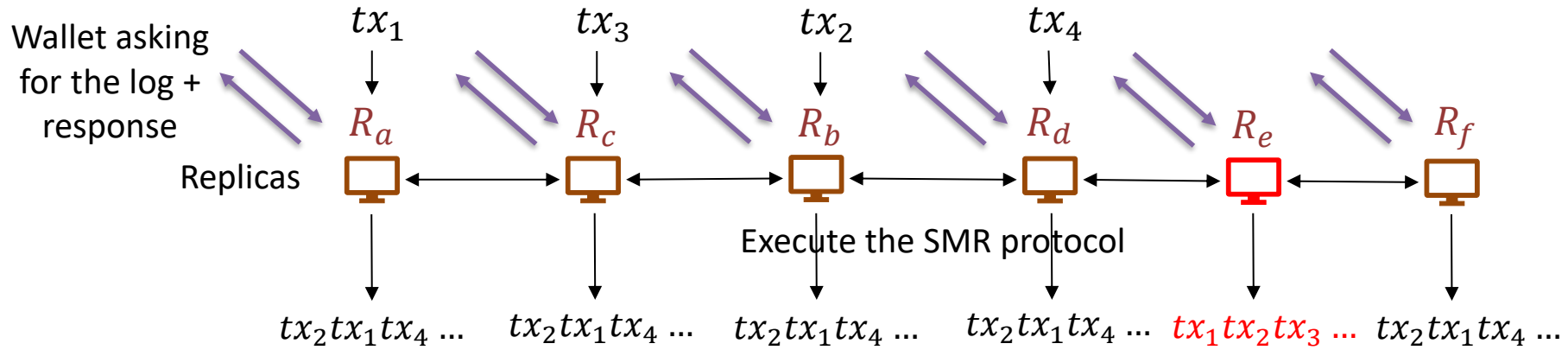


Wallets **do not** execute the SMR protocol and **do not** talk to each other.

$$LOG_t^4 = tx_2tx_1tx_4 \dots$$



State Machine Replication (SMR)



$$LOG_t^1 = tx_2tx_1tx_4 \dots$$

Clients (Wallets)

$$LOG_t^3 = tx_2tx_1tx_4 \dots$$

C_1



How does a wallet learn the correct log from the replicas?

- It asks the replicas what the correct log is.
- Wallet then accepts the answer given by majority of the replicas as its log.

Wallet learns the correct log if over half of the replicas are honest!

$$LOG_t^2 = tx_2tx_1tx_4 \dots$$

Clients (Wallets)

$$LOG_t^4 = tx_2tx_1tx_4 \dots$$

C_2



C_4



Security for SMR: Definitions

Concatenation ($A||B$):

- Suppose we have sequences $A = tx_1tx_2$ and $B = tx_3tx_4$. What is $A||B$?

$$A||B = tx_1tx_2tx_3tx_4$$

Prefix relation ($A \preceq B$): Sequence A is said to be a prefix of sequence B , if there exists a sequence C (that is potentially empty) such that $B = A||C$.

Suppose we have $A = tx_1tx_2tx_3tx_4$, $B = tx_1tx_2tx_3$ and $D = tx_1tx_2tx_4$.

Is B a prefix of A ?

Yes

Is D a prefix of A ?

No

Security for SMR: Definitions

Two sequences A and B are consistent if either $A \preceq B$ is true or $B \preceq A$ is true or both statements are true.

Are these two logs consistent:

$$LOG^{Alice} = tx_1tx_2tx_3tx_4, \quad LOG^{Bob} = tx_1tx_2tx_3?$$

- Yes!

$$\text{What about } LOG^{Alice} = tx_1tx_2tx_3, \quad LOG^{Bob} = tx_1tx_2tx_3tx_4?$$

- Yes!

$$\text{What about } LOG^{Alice} = tx_1tx_2, \quad LOG^{Bob} = tx_1tx_3?$$

- No!

Security for SMR

Let LOG_t^i denote the log output by a client i at time t .

Then, a **secure** SMR protocol satisfies the following guarantees:

Safety (Consistency): Similar to agreement!

- For all clients i and j , and times t and s : either $LOG_t^i \preceq LOG_s^j$ is true or $LOG_s^j \preceq LOG_t^i$ is true or both (Logs are consistent).

Liveness: Similar to validity and termination!

- There is a constant T_{conf} s.t.:
If a transaction tx is input to an honest replica at time t ,
then for all clients i and all times $s \geq t + T_{conf}$ we have $tx \in LOG_s^i$.

Security for SMR

Let LOG_t^i denote the log output by a client i at time t .

Then, a **secure** SMR protocol satisfies the following guarantees:

Safety (Consistency): Similar to agreement!

- For all clients i and j , and times t and s : either $LOG_t^i \preceq LOG_s^j$ is true or $LOG_s^j \preceq LOG_t^i$ is true or both (Logs are consistent).

No double
spend

Liveness: Similar to validity and termination!

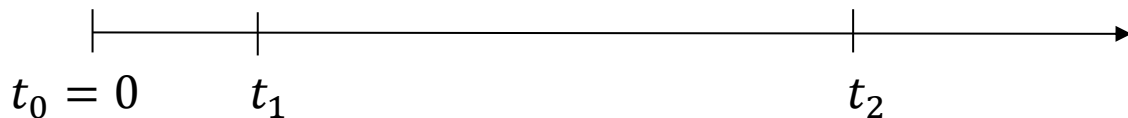
- There is a constant T_{conf} s.t.:
If a transaction tx is input to an honest replica at time t ,
then for all clients i and all times $s \geq t + T_{conf}$ we have $tx \in LOG_s^i$.

No
censorship

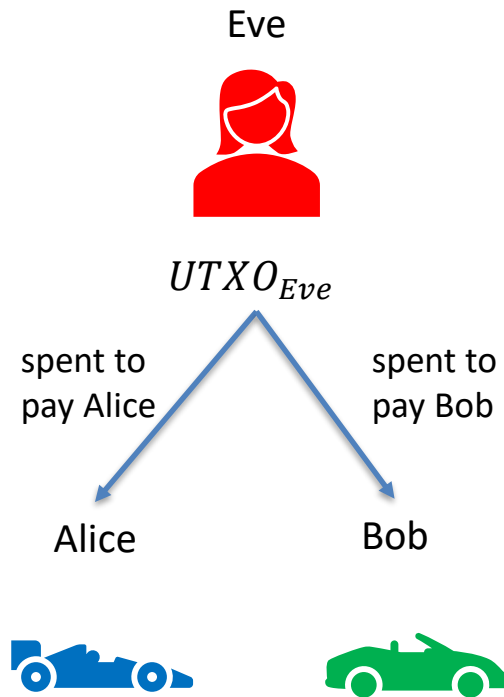
Why is safety important?

Suppose Eve has a UTXO.

- tx_1 : transaction spending Eve's UTXO to pay to car vendor Alice.
- tx_2 : transaction spending Eve's UTXO to pay to car vendor Bob.



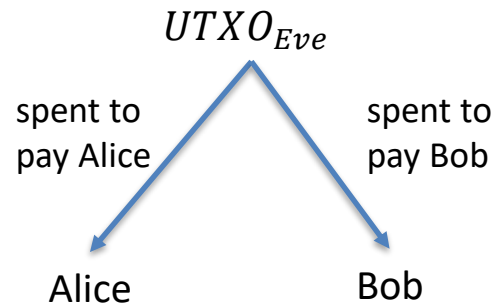
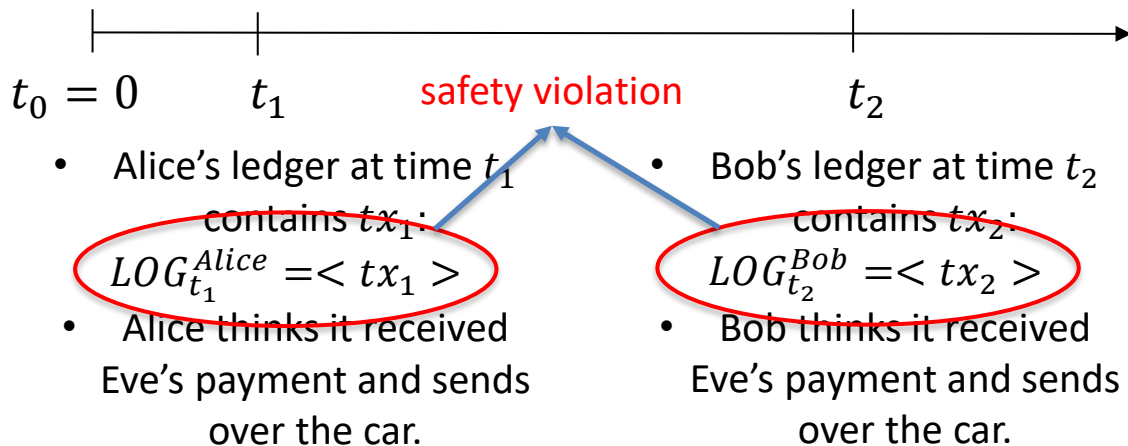
- Alice's ledger at time t_1 contains tx_1 :
 $LOG_{t_1}^{Alice} = \langle tx_1 \rangle$
- Alice thinks it received Eve's payment and sends over the car.
- Bob's ledger at time t_2 contains tx_2 :
 $LOG_{t_2}^{Bob} = \langle tx_2 \rangle$
- Bob thinks it received Eve's payment and sends over the car.



Why is safety important?

Suppose Eve has a UTXO.

- tx_1 : transaction spending Eve's UTXO to pay to car vendor Alice.
- tx_2 : transaction spending Eve's UTXO to pay to car vendor Bob.



When safety is violated, Eve can double-spend!

SMR vs. Byzantine Broadcast

- **Single shot vs. Multi-shot**
 - **Broadcast** is single shot consensus. Each node outputs a single value.
 - **State Machine Replication** is multi-shot. Each client *continuously* outputs a log, which is a sequence of transactions (values).
- **Who are the learners?**
 - In **Broadcast**, the nodes executing the protocol are the same as the nodes that output decision values.
 - In **State Machine Replication**, protocol is executed by the replicas, whereas the goal is for the clients to learn the log.
 - Replicas must ensure that the clients learn the same log.

Let's build an SMR protocol

Next lecture ...

END OF LECTURE

Next lecture: Consensus in the Internet Setting